UCS645: Parallel & Distributed Computing

Assignment 4: Introduction to MPI

Lab Report

Name - Gagandeep

Roll no – 102303349

Exercise 1: Ring Communication

Objective

Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 starts with value 100, and each process adds its rank to the value before forwarding.

Output

Process	Action	Value
Process 0	Starting value	100
Process 1	Received 100, added rank 1, sent to Process 2	101
Process 2	Received 101, added rank 2, sent to Process 3	103
Process 3	Received 103, added rank 3, sent to Process 0	106
Process 0	Final value received back	106

Conclusion

The ring communication was implemented successfully using MPI_Send and MPI_Recv. The final value returned to Process 0 was 106, which is the correct result (100 + 0 + 1 + 2 + 3 = 106). The modulo operator (rank+1) % size ensured proper wrap-around from the last process back to Process 0.

Exercise 2: Parallel Array Sum

Objective

Compute the sum of an array of values 1 to 100 in parallel using MPI_Scatter to distribute and MPI_Reduce to collect results.

Output

Process	Elements Handled	Local Sum
Process 0	1 to 25	325
Process 1	26 to 50	950
Process 2	51 to 75	1575
Process 3	76 to 100	2200

Metric	Value
Global Sum	5050
Expected Sum	5050
Average	50.50
Result	CORRECT

Conclusion

The parallel array sum was computed correctly. Each of the 4 processes handled 25 elements independently, computed their local sum, and MPI_Reduce combined all local sums to produce the correct global sum of 5050 with an average of 50.50.

Exercise 3: Finding Maximum and Minimum

Objective

Each process generates 10 random numbers (0-1000) and the program finds the global maximum and minimum using MPI_MAXLOC and MPI_MINLOC, along with which process held these values.

Output

Process	Generated Numbers	Local Max	Local Min
Process 0	465 161 282 259 114 445 482 714 28 4	714	4
Process 1	155 882 138 826 196 631 203 796 558 519	882	138
Process 2	85 107 876 250 315 93 618 90 952 573	952	85
Process 3	45 493 14 274 340 32 831 440 241 617	831	14

Result	Value	Found in Process
Global Maximum	952	Process 2
Global Minimum	4	Process 0

Conclusion

The program correctly identified the global maximum value of 952 from Process 2 and global minimum value of 4 from Process 0 using MPI_MAXLOC and MPI_MINLOC operations. These operations efficiently find both the extreme value and its location (rank) in a single reduction step.

Exercise 4: Parallel Dot Product

Objective

Compute the dot product of Vector A = [1,2,3,4,5,6,7,8] and Vector B = [8,7,6,5,4,3,2,1] in parallel using MPI_Scatter and MPI_Reduce. Expected result: 120.

Output

Process	Elements of A	Elements of B	Partial Dot Product
Process 0	[1, 2]	[8, 7]	1x8 + 2x7 = 22
Process 1	[3, 4]	[6, 5]	3x6 + 4x5 = 38
Process 2	[5, 6]	[4, 3]	5x4 + 6x3 = 38
Process 3	[7, 8]	[2, 1]	7x2 + 8x1 = 22

Metric	Value
Final Dot Product	120
Expected Result	120
Correct	YES

Conclusion

The parallel dot product was computed correctly. Each process handled 2 elements from both vectors, computed partial products, and MPI_Reduce summed them to produce the correct final result of 120 (22 + 38 + 38 + 22 = 120).

Performance Analysis

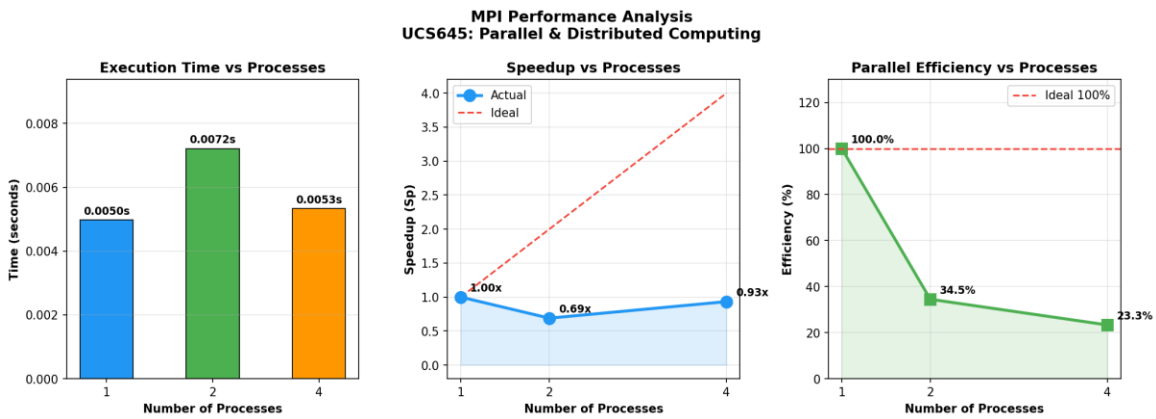
Objective

Measure execution time of the parallel array sum (1,000,000 elements) with 1, 2, and 4 processes to evaluate speedup and efficiency.

Raw Results

Processes (p)	Execution Time Tp (s)	Speedup Sp = T1/Tp	Efficiency Ep = Sp/p x 100%
1	0.004978	1.00	100.00%
2	0.007214	0.69	34.50%
4	0.005334	0.93	23.33%

Performance Graphs



Analysis & Observations

1. Execution Time: With 1 process the time was 0.004978s. With 2 processes it increased to 0.007214s, and with 4 processes it was 0.005334s.

2. Speedup: The speedup was below 1.0 for 2 processes (0.69x) meaning it was actually slower than single process. With 4 processes it recovered slightly to 0.93x but still below ideal.

3. Efficiency: Dropped significantly - 100% with 1 process, 34.5% with 2 processes, and 23.33% with 4 processes.

4. Reason for Poor Scaling: Since this test was run on a VirtualBox with shared CPU cores, the communication overhead (MPI_Scatter and MPI_Reduce over simulated network) outweighs the computation gain for this problem size. On a real multi-node cluster, results would show better speedup.

5. Amdahl's Law: The sequential portion and communication overhead limit parallel gains. For small arrays on a single machine, parallel overhead can exceed computation savings.

Conclusion

The MPI performance test revealed that on a single VirtualBox machine with shared resources, parallelism does not always improve performance for small-to-medium workloads. The communication overhead of MPI_Scatter and MPI_Reduce introduces latency that can exceed computation savings. In a real distributed HPC environment with larger datasets, MPI would demonstrate significant speedup.

The exercises successfully demonstrated all core MPI concepts: point-to-point communication, collective operations (Scatter, Reduce, Broadcast), and performance measurement using MPI_Wtime.